

# Predizione dell'esito delle partite di League of Legends a partire dai primi dieci minuti: un confronto tra modelli di Machine Learning

Melania Leonelli, Junhuang Chen

---

## Abstract

Questo lavoro analizza la possibilità di prevedere l'esito finale di una partita competitiva di League of Legends utilizzando esclusivamente le informazioni disponibili nei primi dieci minuti di gioco.

Lo studio confronta sei algoritmi di Machine Learning su un dataset composto da 9.879 partite ranked di livello Diamond. Il problema viene formulato come un task di classificazione binaria volto a prevedere la variabile `blueWins`, rappresentativa della squadra vincitrice.

La metodologia comprende un'analisi esplorativa dei dati, la gestione della collinearità e delle ridondanze tra le feature, la suddivisione del dataset in Training Set e Test Set e l'applicazione di tecniche di standardizzazione volte a prevenire fenomeni di data leakage.

I classificatori analizzati sono Regressione Logistica, Support Vector Machine, Decision Tree, K-Nearest Neighbors, Random Forest e Gaussian Naive Bayes. Il confronto considera sia le prestazioni predittive sia l'interpretabilità dei modelli, integrando inoltre un'analisi critica degli errori di classificazione. La Random Forest ottiene le migliori prestazioni complessive sul Test Set, con un'Accuracy pari a 0.7247 e un F1-Score pari a 0.7236, mentre la Regressione Logistica raggiunge risultati molto vicini mantenendo una maggiore interpretabilità. L'analisi delle caratteristiche evidenzia inoltre il ruolo centrale dei differenziali di oro ed esperienza nell'early game.

*Keywords:* League of Legends, Machine Learning, Classificazione, eSports, Early-game prediction, Interpretabilità dei modelli

---

## 1. Introduzione

### 1.1. Contesto e motivazione

League of Legends è un videogioco competitivo appartenente al genere Multiplayer Online Battle Arena (MOBA), nel quale due squadre si affrontano con l'obiettivo di ottenere progressivamente un vantaggio strategico e distruggere la struttura principale della squadra avversaria.

Nel corso di una partita, l'esito finale dipende dall'interazione di numerosi fattori. Tra questi rientrano l'accumulo di oro e di esperienza, le eliminazioni, il controllo degli obiettivi neutrali, il farming, la distruzione delle strutture e la gestione della visione della mappa.

La natura fortemente dinamica delle partite rende particolarmente interessante il problema della previsione anticipata del risultato. Determinare già nelle prime fasi di gioco quale squadra presenti una maggiore probabilità di vittoria costituisce infatti un problema di classificazione nel quale le informazioni disponibili sono necessariamente parziali.

Un vantaggio acquisito durante l'early game può influenzare significativamente l'evoluzione della partita, ma non ne determina necessariamente l'esito finale. Eventi successivi, quali rimonte, combattimenti di squadra, conquista di obiettivi o errori individuali, possono modificare il vantaggio accumulato nelle fasi iniziali.

In questo contesto, le tecniche di Machine Learning permettono di studiare quantitativamente la relazione tra lo stato iniziale della partita e il risultato conclusivo.

L'interesse non riguarda soltanto la capacità predittiva dei modelli, ma anche la possibilità di comprendere quali caratteristiche del gioco risultino maggiormente associate alla vittoria e in quali situazioni la previsione risulti maggiormente incerta.

Il presente lavoro affronta pertanto il problema della previsione dell'esito di partite ranked di League of Legends utilizzando esclusivamente le informazioni disponibili al decimo minuto di gioco. Tale scelta permette di valutare quanta informazione predittiva sia già contenuta nell'early game e, contemporaneamente, di studiare i limiti derivanti dall'utilizzo di una rappresentazione necessariamente incompleta dell'intera partita.

### 1.2. Obiettivi del lavoro

L'obiettivo principale dello studio è valutare la possibilità di prevedere l'esito finale di una partita di League of Legends sulla base delle sole informazioni raccolte nei primi dieci minuti di gioco.

A tale scopo viene analizzato un dataset composto da 9.879 partite ranked di livello Diamond.

Il problema viene formulato come un task di classificazione binaria, nel quale la variabile target `blueWins` indica la squadra vincitrice:

- `blueWins` = 1: vittoria del Blue Team;
- `blueWins` = 0: vittoria del Red Team.

Il lavoro confronta sei differenti algoritmi di Machine Learning:

- Regressione Logistica;
- Support Vector Machine;
- Decision Tree;
- K-Nearest Neighbors;
- Random Forest;
- Gaussian Naive Bayes.

La scelta di modelli appartenenti a famiglie differenti consente di confrontare approcci lineari, geometrici, basati sulla distanza, probabilistici e ad albero.

L'analisi non si limita al confronto delle prestazioni predittive. Vengono inoltre studiate l'importanza attribuita alle diverse feature, la capacità interpretativa dei modelli e le caratteristiche delle partite classificate erroneamente.

L'obiettivo è quindi valutare non soltanto quale classificatore generalizzi meglio su dati non utilizzati durante l'addestramento, ma anche quali informazioni dell'early game risultino maggiormente associate alla previsione dell'esito della partita.

### 1.3. Domande di ricerca

Alla luce degli obiettivi descritti, il presente studio è guidato dalle seguenti domande di ricerca:

- RQ1.** In quale misura le informazioni relative alle risorse accumulate e allo stato della partita nei primi dieci minuti di gioco contengono un segnale predittivo utile per anticipare l'esito finale di un match ranked di livello Diamond?
- RQ2.** Quale compromesso emerge tra le prestazioni predittive dei modelli più complessi, come la Random Forest, e la maggiore interpretabilità di modelli quali la Regressione Logistica e il Decision Tree?
- RQ3.** Quali caratteristiche distinguono le partite classificate erroneamente e in quale misura gli errori risultano associati a match economicamente più equilibrati al decimo minuto?

## 2. Stato dell'arte

### 2.1. Machine Learning nei MOBA e negli eSports

La crescente disponibilità di dati relativi alle competizioni videoludiche ha favorito l'applicazione di tecniche di Data Mining e Machine Learning all'analisi degli eSports. In particolare, i giochi appartenenti al genere Multiplayer Online Battle Arena (MOBA), come League of Legends e Dota 2, costituiscono un dominio particolarmente adatto allo studio di problemi predittivi, poiché ogni partita produce una grande quantità di informazioni relative alle risorse accumulate, alle eliminazioni, al controllo degli obiettivi, al farming e alle interazioni tra i giocatori. Diversi studi hanno infatti applicato tecniche di Machine Learning alla previsione dell'esito delle partite di League of Legends utilizzando differenti tipologie di informazioni disponibili prima o durante il match (Do et al., 2021; Junior and Campelo, 2023; Bahrololloomi et al., 2023).

Dal punto di vista metodologico, tali applicazioni possono essere ricondotte al più generale processo di Knowledge Discovery in Databases (KDD), nel quale l'addestramento di un algoritmo rappresenta soltanto una delle fasi di un processo più ampio comprendente la selezione dei dati, la loro preparazione, l'individuazione di pattern e la successiva interpretazione dei risultati (Fayyad et al., 1996).

Il problema affrontato nel presente studio può inoltre essere descritto secondo la tradizionale formulazione dell'apprendimento automatico basata su esperienza, task e misura di performance (Mitchell, 1997). L'esperienza è rappresentata dallo storico delle partite disponibili nel dataset; il task consiste nella classificazione binaria dell'esito del match mediante la variabile `blueWins`; le prestazioni vengono infine misurate attraverso metriche di classificazione quali Accuracy, Precision, Recall e F1-Score.

Nella letteratura sulla predizione dell'esito in League of Legends emergono approcci che operano in momenti differenti del match. Alcuni studi formulano la previsione prima dell'inizio della partita, utilizzando informazioni sui giocatori e sui campioni selezionati. Ad esempio, Do et al. (2021) utilizzano l'esperienza dei giocatori con i rispettivi campioni per prevedere il risultato prima dell'inizio del gameplay.

Altri lavori adottano invece un approccio di *in-game prediction*, utilizzando informazioni raccolte durante lo svolgimento della partita. Junior and Campelo (2023), ad esempio, analizzano la capacità di diversi algoritmi di prevedere l'esito di League of Legends in differenti momenti del match, mostrando come le prestazioni dei modelli cambino in funzione della quantità di informazione disponibile.

Il presente lavoro si colloca in quest'ultimo filone, poiché utilizza esclusivamente le informazioni disponibili al decimo minuto di gioco. La previsione viene quindi effettuata in una fase relativamente precoce del match, quando una parte delle dinamiche rilevanti per l'esito finale non si è ancora manifestata.

Questa caratteristica rende la previsione early-game particolarmente interessante. Variabili quali il differenziale di oro, il differenziale di esperienza, le eliminazioni o gli obiettivi conquistati possono contenere già nelle prime fasi della partita un segnale associato all'esito finale. Tuttavia, tali informazioni rappresentano soltanto una fotografia parziale del match.

Eventi successivi al momento dell'osservazione, come combattimenti di squadra, rimonte, conquista di obiettivi tardivi o errori individuali, non sono rappresentati nelle feature utilizzate dal classificatore. La previsione dell'esito al minuto 10 costituisce pertanto un problema nel quale il modello deve generalizzare a partire da informazioni necessariamente incomplete.

L'obiettivo di questo studio non è quindi soltanto ottenere la massima capacità predittiva possibile, ma anche verificare quanta informazione utile sull'esito finale sia già contenuta nello stato dell'early game e quali caratteristiche risultino maggiormente associate alla vittoria.

## 2.2. Il compromesso tra interpretabilità e prestazioni

Un tema centrale nell'applicazione del Machine Learning riguarda il rapporto tra capacità predittiva, complessità del modello e interpretabilità. La distinzione tra modelli intrinsecamente interpretabili e modelli la cui funzione decisionale risulta più difficile da comprendere costituisce un tema ampiamente discusso nella letteratura sul Machine Learning interpretabile (Rudin, 2019).

Modelli differenti possono infatti raggiungere prestazioni simili utilizzando rappresentazioni della funzione decisionale profondamente diverse. Da un lato si collocano modelli caratterizzati da una maggiore trasparenza, per i quali è possibile analizzare più direttamente il meccanismo che conduce alla classificazione. Nel presente studio, la Regressione Logistica e il Decision Tree rappresentano gli esempi principali di questa categoria.

Nella Regressione Logistica la decisione viene ottenuta attraverso una combinazione lineare delle feature:

$$z = \beta_0 + \sum_{i=1}^p \beta_i x_i, \quad (1)$$

successivamente trasformata mediante la funzione sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (2)$$

I coefficienti  $\beta_i$  consentono di osservare la direzione e l'intensità dell'associazione tra ciascuna variabile e i log-odds della classe positiva, mantenendo costanti le altre feature. Tale proprietà rende la Regressione Logistica particolarmente utile quando, oltre alla predizione, è necessario comprendere il contributo delle singole caratteristiche.

Il Decision Tree offre invece una forma di interpretabilità basata su una struttura gerarchica di decisioni. Ogni nodo interno rappresenta una condizione applicata a una feature, mentre il percorso dalla radice a una foglia può essere espresso sotto forma di regola logica *IF-THEN*. Questa struttura permette di rappresentare in maniera diretta alcune delle condizioni che conducono il classificatore a prevedere la vittoria del Blue Team o del Red Team.

All'aumentare della complessità del modello, tuttavia, l'interpretazione diretta della funzione decisionale può diventare più difficile. La Random

Forest, ad esempio, combina le predizioni di numerosi alberi di decisione addestrati introducendo variabilità sia nella selezione dei campioni sia nella selezione delle feature considerate durante la costruzione degli alberi (Breiman, 2001).

L'aggregazione di più classificatori permette di ridurre la sensibilità alle variazioni del Training Set tipica di un singolo albero e può migliorare la capacità di generalizzazione. Il prezzo di tale strategia è una minore trasparenza: non è più possibile descrivere la decisione complessiva attraverso un singolo insieme compatto di regole.

L'interpretazione di un modello ensemble può quindi essere supportata da strumenti globali come la *Feature Importance*, che permette di valutare quali variabili contribuiscano maggiormente alle decisioni del modello, pur senza ricostruire completamente il processo decisionale delle singole predizioni.

Il confronto tra modelli semplici e modelli più complessi assume quindi un ruolo centrale nel presente studio. L'obiettivo non consiste soltanto nell'individuare il classificatore con le migliori prestazioni sul Test Set, ma anche nel valutare se l'eventuale incremento delle prestazioni predittive giustifichi la perdita di interpretabilità associata a modelli ensemble più complessi.

Questo confronto consente inoltre di verificare se, nel dominio considerato, una rappresentazione relativamente semplice delle relazioni tra le risorse accumulate nei primi dieci minuti sia già sufficiente a ottenere prestazioni competitive oppure se siano necessarie interazioni non lineari più complesse per migliorare la previsione dell'esito finale.

### 3. Materiali e Metodi

#### 3.1. Dataset e variabili di gioco

Il dataset utilizzato nel presente studio contiene informazioni relative a 9.879 partite ranked di League of Legends disputate da giocatori di livello Diamond. Ciascuna osservazione corrisponde a una singola partita e rappresenta lo stato del match esattamente al decimo minuto di gioco, fornendo una fotografia delle risorse accumulate e delle principali azioni compiute dalle due squadre durante la fase iniziale della partita.

Il dataset originale è composto da 9.879 osservazioni e 40 colonne. L'analisi preliminare dei dati non ha evidenziato valori mancanti né record duplicati.

La variabile target è `blueWins`, una variabile binaria che identifica la squadra vincitrice:

- `blueWins = 1`: vittoria del Blue Team;
- `blueWins = 0`: vittoria del Red Team.

La distribuzione della variabile target risulta quasi perfettamente bilanciata. Il Blue Team vince 4.930 partite, pari a circa il 49,90% del dataset, mentre il Red Team vince 4.949 partite, pari a circa il 50,10%.

La sostanziale equivalenza delle frequenze delle due classi indica che il dataset non presenta un problema rilevante di *class imbalance*. Di conseguenza, l'Accuracy può essere utilizzata come una misura informativa delle prestazioni complessive dei classificatori, pur venendo affiancata da Precision, Recall e F1-Score per ottenere una valutazione più completa del comportamento dei modelli.

Tra le 40 colonne è inoltre presente `gameId`, che rappresenta esclusivamente l'identificativo univoco della partita e non contiene informazioni relative allo stato competitivo del match. Per tale ragione questa variabile viene successivamente esclusa dall'insieme delle feature utilizzate per l'addestramento.

Escludendo `gameId` e la variabile target `blueWins`, il dataset originale contiene quindi 38 feature predittive. La maggior parte di esse è disponibile in forma simmetrica per il Blue Team e per il Red Team e può essere ricondotta a sei principali categorie: economia, esperienza, combattimento, farming, obiettivi e visione della mappa.



Tabella 1: Principali categorie delle feature presenti nel dataset.

| Categoria     | Variabili principali                                         | Descrizione                                                                                                                                                                                         |
|---------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Economia      | TotalGold,<br>GoldPerMin,<br>GoldDiff                        | Risorse economiche accumulate dalle squadre, ritmo di acquisizione dell'oro e differenziale economico tra Blue Team e Red Team.                                                                     |
| Esperienza    | TotalExperience,<br>AvgLevel,<br>ExperienceDiff              | Esperienza complessivamente acquisita, livello medio dei giocatori e differenziale di esperienza tra le due squadre.                                                                                |
| Combattimento | Kills, Deaths,<br>Assists, FirstBlood                        | Informazioni relative agli scontri tra giocatori: eliminazioni effettuate, morti subite, assist e ottenimento del First Blood.                                                                      |
| Farming       | TotalMinionsKilled,<br>TotalJungleMinionsKilled,<br>CSPerMin | Numero di minion eliminati, mostri neutrali della giungla eliminati e ritmo di farming espresso in minion uccisi al minuto.                                                                         |
| Obiettivi     | Dragons, Heralds,<br>EliteMonsters,<br>TowersDestroyed       | Controllo degli obiettivi della mappa attraverso Draghi, Rift Herald e torri distrutte. EliteMonsters rappresenta una variabile aggregata derivata dalla somma di Draghi e Rift Herald conquistati. |
| Visione       | WardsPlaced,<br>WardsDestroyed                               | Controllo informativo della mappa attraverso il posizionamento delle ward e la distruzione delle ward avversarie.                                                                                   |

Nel dataset i nomi delle feature sono preceduti dal prefisso **blue** o **red** a seconda della squadra a cui si riferiscono. Ad esempio, **blueTotalGold** rappresenta l'oro totale accumulato dal Blue Team al decimo minuto, mentre **redTotalGold** rappresenta la corrispondente quantità per il Red Team.

Alcune variabili descrivono direttamente il divario tra le due squadre. In particolare, **blueGoldDiff** rappresenta il differenziale di oro dal punto di vista del Blue Team, mentre **redGoldDiff** rappresenta lo stesso diva-

rio dal punto di vista opposto. Relazioni di questo tipo introducono ridondanze strutturali che verranno analizzate e gestite nella successiva fase di pre-elaborazione e selezione delle feature.

Nel complesso, il dataset permette di rappresentare differenti dimensioni dello stato della partita al minuto 10. Le variabili economiche e di esperienza descrivono il vantaggio accumulato dalle squadre, mentre le informazioni relative a combattimento, farming, obiettivi e visione contribuiscono a descrivere le modalità attraverso cui tale stato di gioco è stato raggiunto.

### *3.2. Analisi esplorativa dei dati e qualità del dato*

L'analisi esplorativa dei dati (Exploratory Data Analysis, EDA) è stata condotta prima della fase di preprocessing e modellazione con l'obiettivo di verificare la struttura e la qualità del dataset, analizzare le distribuzioni delle principali variabili e individuare eventuali valori anomali o caratteristiche potenzialmente rilevanti per la successiva costruzione dei modelli.

#### *3.2.1. Profiling e qualità dei dati*

La fase di data profiling consente di esaminare sistematicamente la struttura, la completezza e le principali proprietà statistiche dei dati prima dell'applicazione delle tecniche di Machine Learning (?).

L'ispezione preliminare ha confermato che il dataset è composto da 9.879 osservazioni e 40 colonne. Tutte le variabili sono rappresentate in formato numerico: 34 colonne sono di tipo `int64`, mentre 6 sono di tipo `float64`. Queste ultime corrispondono alle variabili `AvgLevel`, `CSPerMin` e `GoldPerMin`, presenti separatamente per Blue Team e Red Team.

Il controllo della completezza dei dati non ha evidenziato valori mancanti: ciascuna delle 40 colonne presenta infatti 9.879 osservazioni valide e il numero complessivo di valori nulli è pari a zero. Non è stato quindi necessario applicare procedure di imputazione prima della modellazione.

È stata inoltre verificata l'eventuale presenza di duplicati. Il controllo sulle righe complete non ha individuato osservazioni duplicate e la verifica separata della variabile `gameId` ha confermato l'assenza di identificativi ripetuti. Ciascuna osservazione del dataset è quindi associata a un diverso identificativo di partita.

Le statistiche descrittive evidenziano valori medi molto simili tra le due squadre per diverse variabili di gioco. Ad esempio, al decimo minuto il Blue Team registra in media circa 6,18 eliminazioni, mentre il Red Team ne registra circa 6,14. Le rispettive deviazioni standard sono circa 3,01 e 2,93.

Una situazione analoga emerge per le risorse economiche. L'oro totale medio al decimo minuto è pari a circa 16.503 per il Blue Team e 16.489 per il Red Team. Tale vicinanza tra i valori aggregati delle due squadre è coerente con un dataset nel quale non emerge, a livello descrittivo generale, un forte vantaggio sistematico iniziale attribuibile a una delle due fazioni.

### 3.2.2. Distribuzioni e valori anomali

L'analisi delle distribuzioni è stata effettuata mediante rappresentazioni grafiche, tra cui istogrammi e boxplot, con particolare attenzione alle variabili che descrivono il vantaggio tra le due squadre e alle feature caratterizzate dalla presenza di valori estremi.

Gli istogrammi di `blueGoldDiff` e `blueExperienceDiff` consentono di osservare come i differenziali di oro e di esperienza siano distribuiti attorno a valori prossimi allo zero. La maggior parte delle osservazioni si concentra nella regione centrale della distribuzione, mentre differenziali di maggiore entità sono meno frequenti e costituiscono le code della distribuzione.

La presenza di valori potenzialmente anomali è stata successivamente analizzata attraverso i boxplot e mediante il metodo dell'Interquartile Range (IQR). Definendo:

$$IQR = Q_3 - Q_1, \quad (3)$$

sono stati considerati potenziali outlier i valori esterni all'intervallo:

$$[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]. \quad (4)$$

L'applicazione di tale criterio ad alcune variabili selezionate ha evidenziato:

- 88 potenziali outlier per `blueKills` e 65 per `redKills`, con limiti IQR pari a  $[-2, 14]$ ;
- 1.627 potenziali outlier per `blueWardsPlaced` e 1.667 per `redWardsPlaced`, con limiti IQR pari a  $[5, 29]$ ;
- 136 potenziali outlier sia per `blueWardsDestroyed` sia per `redWardsDestroyed`, con limiti IQR pari a  $[-3.5, 8.5]$ .

Nel caso delle eliminazioni, il valore massimo osservato è pari a 22 kills per entrambe le squadre. Le variabili relative alle ward presentano invece valori

estremi particolarmente elevati: `blueWardsPlaced` raggiunge un massimo di 250, mentre `redWardsPlaced` raggiunge un massimo di 276.

È importante sottolineare che il criterio IQR identifica valori statisticamente distanti dalla parte centrale della distribuzione, ma non consente da solo di stabilire che tali osservazioni siano errori di rilevazione. In assenza di evidenze sufficienti per classificare questi valori come dati errati, si è scelto di non rimuovere automaticamente le osservazioni identificate come potenziali outlier.

Questa scelta consente di preservare la variabilità presente nel dataset originale ed evita di modificare arbitrariamente la distribuzione delle variabili sulla sola base della loro rarità. I valori estremi vengono pertanto mantenuti nelle successive fasi dell'analisi.

### 3.2.3. Bilanciamento della variabile target

Poiché il problema analizzato è formulato come una classificazione binaria, durante l'analisi esplorativa è stata verificata anche la distribuzione della variabile target `blueWins`.

Nel dataset si osservano:

- 4.949 osservazioni appartenenti alla classe `blueWins = 0`, corrispondenti a una vittoria del Red Team e pari al 50,10% circa del dataset;
- 4.930 osservazioni appartenenti alla classe `blueWins = 1`, corrispondenti a una vittoria del Blue Team e pari al 49,90% circa del dataset.

La variabile target risulta pertanto sostanzialmente bilanciata, con una differenza di appena 19 osservazioni tra le due classi. Non emerge quindi un problema rilevante di *class imbalance*.

Questa caratteristica permette di interpretare l'Accuracy senza il problema tipico dei dataset fortemente sbilanciati, nei quali un classificatore potrebbe ottenere un valore apparentemente elevato limitandosi a predire prevalentemente la classe maggioritaria.

L'Accuracy non viene tuttavia utilizzata come unico criterio di valutazione. Nella fase sperimentale viene affiancata da Precision, Recall e F1-Score, in modo da analizzare più dettagliatamente il comportamento di ciascun classificatore e le differenti tipologie di errore commesse.

### 3.3. Pre-elaborazione e selezione delle feature

A partire dalle evidenze emerse durante l'analisi esplorativa, è stata definita una pipeline di pre-elaborazione finalizzata a ridurre la ridondanza informativa del dataset e a preparare le variabili per l'addestramento dei classificatori.

La procedura ha previsto la rimozione delle feature fortemente collineari o funzionalmente dipendenti da altre variabili, l'esclusione dell'identificativo `gameId`, la separazione della variabile target `blueWins`, la suddivisione delle osservazioni in Training Set e Test Set e, infine, la standardizzazione delle feature numeriche.

Al termine della selezione, le 38 variabili candidate originariamente disponibili, escludendo `gameId` e `blueWins`, sono state ridotte a 27 feature effettivamente utilizzate per la modellazione.

#### 3.3.1. Analisi di collinearità e ridondanza

L'analisi delle relazioni tra le variabili ha evidenziato la presenza di feature che rappresentano informazioni fortemente sovrapposte o, in alcuni casi, matematicamente dipendenti da altre variabili già presenti nel dataset.

La conservazione simultanea di tali feature non avrebbe apportato nuova informazione predittiva e avrebbe aumentato inutilmente la dimensionalità del problema. Tale aspetto è particolarmente rilevante per i modelli lineari, nei quali la presenza di forte multicollinearità può rendere più difficile l'interpretazione dei coefficienti.

Una prima operazione ha riguardato la rimozione delle seguenti variabili:

- `blueCSPerMin` e `redCSPerMin`, in quanto fortemente ridondanti rispetto alle informazioni di farming già rappresentate nel dataset;
- `blueAssists` e `redAssists`, che presentano una forte sovrapposizione informativa con altre variabili relative al combattimento, all'oro e all'esperienza.

Una seconda forma di ridondanza deriva dalla struttura stessa del dataset, nel quale alcune feature del Red Team rappresentano esattamente la stessa informazione già espressa dal punto di vista del Blue Team.

In particolare, valgono le seguenti relazioni:

$$\text{redFirstBlood} = 1 - \text{blueFirstBlood}, \quad (5)$$

$$\text{redKills} = \text{blueDeaths}, \quad (6)$$

$$\text{redDeaths} = \text{blueKills}, \quad (7)$$

$$\text{redGoldDiff} = -\text{blueGoldDiff}, \quad (8)$$

$$\text{redExperienceDiff} = -\text{blueExperienceDiff}. \quad (9)$$

Per evitare di rappresentare due volte la medesima informazione, sono state quindi eliminate le feature `redFirstBlood`, `redKills`, `redDeaths`, `redGoldDiff` e `redExperienceDiff`, mantenendo la corrispondente rappresentazione dal punto di vista del Blue Team.

Infine, sono state eliminate anche le variabili `blueEliteMonsters` e `redEliteMonsters`. Queste feature sono infatti aggregazioni esatte degli obiettivi già rappresentati separatamente attraverso `Draghi` e `Rift Herald`:

$$\text{blueEliteMonsters} = \text{blueDragons} + \text{blueHeralds}, \quad (10)$$

$$\text{redEliteMonsters} = \text{redDragons} + \text{redHeralds}. \quad (11)$$

Sono state mantenute le variabili relative a `Draghi` e `Rift Herald` separatamente, poiché descrivono obiettivi di gioco differenti e conservano quindi un maggiore livello di dettaglio.

Complessivamente, questa procedura ha comportato l'eliminazione di 11 colonne ridondanti. Il dataset è così passato da 40 a 29 colonne, comprendendo ancora la variabile target `blueWins` e l'identificativo `gameId`.

Successivamente, `gameId` è stato eliminato poiché costituisce esclusivamente un identificativo della partita e non una caratteristica predittiva. La variabile `blueWins` è stata infine separata dalle feature e utilizzata come target  $y$ . L'insieme finale delle variabili predittive  $X$  risulta pertanto composto da 27 feature.

### 3.3.2. Partizionamento Training/Test

Dopo la selezione delle feature, il dataset è stato suddiviso in Training Set e Test Set mediante la funzione `train_test_split` di scikit-learn (?).

È stata adottata una suddivisione 80/20:

$$80\% \quad \text{Training Set}, \quad 20\% \quad \text{Test Set}. \quad (12)$$

Il Training Set è utilizzato per l'addestramento e la selezione dei modelli, mentre il Test Set viene mantenuto separato e utilizzato esclusivamente per la valutazione finale delle loro prestazioni su osservazioni non viste durante l'addestramento.

La suddivisione è stata implementata impostando `random_state=42`, al fine di garantire la riproducibilità dell'esperimento, e utilizzando l'opzione `stratify=y`. La stratificazione permette di preservare nei due sottoinsiemi la distribuzione sostanzialmente bilanciata della variabile target `blueWins`.

Il Test Set non è stato utilizzato per la selezione delle feature, l'ottimizzazione degli iperparametri o altre decisioni relative all'addestramento dei modelli.

### 3.3.3. Standardizzazione Z-Score

Per gli algoritmi che utilizzano feature standardizzate è stato applicato lo `StandardScaler` di scikit-learn (?).

La standardizzazione trasforma ciascuna osservazione secondo la formula:

$$z = \frac{x - \mu}{\sigma}, \quad (13)$$

dove  $x$  rappresenta il valore originale della variabile,  $\mu$  la media e  $\sigma$  la deviazione standard.

In questo modo le feature vengono espresse su una scala confrontabile, con media prossima a zero e deviazione standard unitaria. La standardizzazione assume particolare rilevanza per algoritmi sensibili alla scala delle variabili, come K-Nearest Neighbors e Support Vector Machine, poiché impedisce alle feature caratterizzate da valori numericamente più elevati di dominare artificialmente il calcolo delle distanze o la costruzione della superficie decisionale.

Un aspetto metodologico fondamentale riguarda la prevenzione del *data leakage*. Lo `StandardScaler` è stato infatti addestrato esclusivamente sul Training Set:

```
scaler.fit_transform(X_train)
```

Le statistiche ottenute dal Training Set sono state successivamente utilizzate per trasformare il Test Set:

```
scaler.transform(X_test)
```

La media e la deviazione standard del Test Set non vengono quindi utilizzate durante il preprocessing. In questo modo nessuna informazione proveniente dalle osservazioni destinate alla valutazione finale può influenzare la preparazione dei dati utilizzati per l'addestramento.

### 3.4. Modelli di Machine Learning

Per affrontare il problema di classificazione sono stati considerati sei algoritmi di Machine Learning appartenenti a famiglie differenti: Regressione Logistica, Support Vector Machine, Decision Tree, K-Nearest Neighbors, Random Forest e Gaussian Naive Bayes.

La scelta permette di confrontare modelli lineari, metodi basati sul margine, classificatori ad albero, algoritmi basati sulla distanza, metodi ensemble e classificatori probabilistici (??).

Per ciascun algoritmo, l'addestramento e l'eventuale selezione degli iperparametri sono stati eseguiti esclusivamente sul Training Set. Il Test Set è stato mantenuto separato per la successiva valutazione finale.

#### 3.4.1. Regressione Logistica

La Regressione Logistica è un modello di classificazione lineare che stima la probabilità di appartenenza alla classe positiva a partire da una combinazione lineare delle feature.

Indicando con  $x_1, \dots, x_p$  le variabili predittive, il modello calcola inizialmente:

$$z = \beta_0 + \sum_{i=1}^p \beta_i x_i, \quad (14)$$

dove  $\beta_0$  rappresenta l'intercetta e  $\beta_i$  i coefficienti associati alle singole feature.

Il valore ottenuto viene trasformato mediante la funzione sigmoide:

$$P(Y = 1 \mid X) = \frac{1}{1 + e^{-z}}. \quad (15)$$

Nel presente problema, la classe positiva corrisponde a `blueWins = 1`. Un coefficiente positivo indica quindi un'associazione con una maggiore probabilità di vittoria del Blue Team, mentre un coefficiente negativo è associato alla classe opposta.



Oltre alla capacità predittiva, la Regressione Logistica presenta il vantaggio di consentire l'analisi diretta dei coefficienti, rendendola particolarmente utile ai fini dell'interpretabilità.

Il modello è stato addestrato sul Training Set standardizzato. La selezione degli iperparametri è stata effettuata mediante Grid Search con 5-Fold Stratified Cross-Validation, considerando differenti livelli di regolarizzazione. La configurazione selezionata presenta  $C = 0.01$  e regolarizzazione L1.

Il parametro  $C$  controlla la forza della regolarizzazione: valori inferiori di  $C$  corrispondono a una penalizzazione più intensa, utile per limitare la complessità del modello e ridurre il rischio di overfitting.

### 3.4.2. Support Vector Machine

La Support Vector Machine (SVM) costruisce una superficie decisionale con l'obiettivo di separare le osservazioni appartenenti alle due classi massimizzando il margine tra esse.

Nel caso lineare, la funzione decisionale può essere rappresentata come:

$$f(X) = \mathbf{w}^T X + b, \quad (16)$$

dove  $\mathbf{w}$  rappresenta il vettore dei pesi e  $b$  il termine di intercetta. La classe prevista dipende dal lato dell'iperpiano in cui si trova l'osservazione.

Le osservazioni più vicine alla superficie di separazione, definite *support vectors*, contribuiscono in maniera determinante alla definizione del margine.

Il parametro  $C$  regola il compromesso tra ampiezza del margine e penalizzazione degli errori di classificazione. Valori elevati di  $C$  attribuiscono maggiore importanza alla corretta classificazione delle osservazioni di training, mentre valori più bassi consentono un margine più ampio e una maggiore regolarizzazione.

Nella Grid Search sono stati confrontati un kernel lineare e un kernel Radial Basis Function (RBF), per valori di  $C$  pari a 0.01, 0.1, 1, 10 e 100.

La configurazione selezionata utilizza:

- kernel lineare;
- $C = 0.01$ .

La SVM è stata addestrata sul Training Set standardizzato, poiché la scala delle feature può influenzare significativamente la geometria dello spazio utilizzato dal classificatore.

### 3.4.3. Decision Tree

Il Decision Tree è un classificatore che suddivide ricorsivamente lo spazio delle feature attraverso una sequenza di condizioni decisionali.

A ogni nodo interno viene selezionata una feature e una soglia che permettano di separare le osservazioni in gruppi progressivamente più omogenei rispetto alla variabile target. Le foglie terminali rappresentano infine le classi previste.

Nel presente studio sono stati confrontati due criteri per misurare l'impurità dei nodi: indice di Gini ed Entropia.

L'indice di Gini può essere espresso come:

$$G = 1 - \sum_{k=1}^K p_k^2, \quad (17)$$

dove  $p_k$  rappresenta la proporzione delle osservazioni appartenenti alla classe  $k$  nel nodo.

L'Entropia è invece definita come:

$$H = - \sum_{k=1}^K p_k \log_2(p_k). \quad (18)$$

In entrambi i casi, valori più bassi indicano nodi caratterizzati da una maggiore purezza.

La Grid Search ha esplorato differenti combinazioni di criterio di impurità, profondità massima dell'albero e numero minimo di osservazioni necessarie per effettuare uno split. La configurazione selezionata è:

- criterio: `gini`;
- `max_depth = 3`;
- `min_samples_split = 2`.

A differenza degli algoritmi sensibili alla scala, il Decision Tree è stato addestrato sul Training Set non standardizzato. Le divisioni dell'albero dipendono infatti dall'ordinamento delle singole feature e non dalla loro scala numerica.

L'utilizzo delle unità originali facilita inoltre l'interpretazione delle regole estratte: ad esempio, una soglia applicata a `blueGoldDiff` può essere interpretata direttamente come una quantità di oro.

Ogni percorso dalla radice a una foglia può essere espresso sotto forma di regola *IF-THEN*, caratteristica che rende il Decision Tree uno dei modelli maggiormente interpretabili considerati nello studio.

#### 3.4.4. *K-Nearest Neighbors*

K-Nearest Neighbors (KNN) è un algoritmo non parametrico che classifica una nuova osservazione sulla base delle classi delle  $k$  osservazioni più vicine nello spazio delle feature.

La nozione di vicinanza dipende dalla metrica di distanza utilizzata. Nella sperimentazione sono state confrontate la distanza Euclidea e la distanza Manhattan.

La distanza Euclidea tra due osservazioni  $x$  e  $y$  è definita come:

$$d_E(x, y) = \sqrt{\sum_{i=1}^p (x_i - y_i)^2}, \quad (19)$$

mentre la distanza Manhattan è:

$$d_M(x, y) = \sum_{i=1}^p |x_i - y_i|. \quad (20)$$

La Grid Search ha confrontato differenti valori del numero di vicini  $k$ , le due metriche di distanza e due strategie di pesatura: **uniform** e **distance**.

La configurazione selezionata utilizza:

- `n_neighbors = 15`;
- distanza Manhattan;
- pesatura uniforme dei vicini.

Poiché KNN basa direttamente la classificazione sulle distanze tra osservazioni, il modello è stato addestrato sul Training Set standardizzato. Senza scaling, feature caratterizzate da scale numericamente più ampie, come l'oro, potrebbero dominare artificialmente il calcolo della distanza.

### 3.4.5. *Random Forest*

La Random Forest è un metodo ensemble che combina le predizioni di numerosi alberi di decisione (Breiman, 2001).

Ogni albero viene costruito utilizzando un campione dei dati di training ottenuto tramite bootstrap e considerando un sottoinsieme casuale delle feature durante la selezione degli split. La predizione finale viene ottenuta aggregando le decisioni dei singoli alberi, tipicamente mediante voto di maggioranza nel caso della classificazione.

Questa strategia permette di ridurre la varianza tipica di un singolo Decision Tree e di costruire un modello meno sensibile alle variazioni del campione di training.

La Grid Search ha ottimizzato tre principali iperparametri:

- il numero di alberi della foresta (`n_estimators`);
- la profondità massima degli alberi (`max_depth`);
- il numero di feature candidate considerate in ciascuno split (`max_features`).

La configurazione selezionata è:

- `n_estimators = 100`;
- `max_depth = 5`;
- `max_features = log2`.

Nel presente esperimento il modello è stato addestrato utilizzando il Training Set standardizzato, mantenendo così la stessa rappresentazione dei dati utilizzata per gli altri classificatori presenti nello stesso notebook. La standardizzazione non costituisce tuttavia un requisito teorico della Random Forest, poiché gli alberi operano attraverso soglie applicate alle singole feature.

La Random Forest permette inoltre di ottenere una misura globale dell'importanza delle caratteristiche (*Feature Importance*), utilizzata successivamente per analizzare il contributo relativo delle variabili alla previsione dell'esito.

### 3.4.6. Gaussian Naive Bayes

Gaussian Naive Bayes è un classificatore probabilistico basato sul Teorema di Bayes.

Per una classe  $Y$  e un insieme di feature  $X$ , la probabilità a posteriori è espressa come:

$$P(Y | X) = \frac{P(X | Y)P(Y)}{P(X)}. \quad (21)$$

Nel presente studio  $Y$  corrisponde alla variabile target **blueWins**, mentre  $X$  rappresenta le caratteristiche dello stato della partita al decimo minuto.

L'elemento caratterizzante del Naive Bayes è l'assunzione di indipendenza condizionata tra le feature, una volta nota la classe. Formalmente:

$$P(x_1, \dots, x_p | Y) = \prod_{i=1}^p P(x_i | Y). \quad (22)$$

Tale assunzione costituisce una semplificazione importante nel dominio analizzato, poiché variabili come oro, esperienza, eliminazioni e farming possono presentare relazioni reciproche.

Nella variante Gaussian Naive Bayes si assume inoltre che, all'interno di ciascuna classe, le feature numeriche possano essere modellate attraverso una distribuzione Gaussiana:

$$P(x_i | Y = c) = \frac{1}{\sqrt{2\pi\sigma_{ic}^2}} \exp\left(-\frac{(x_i - \mu_{ic})^2}{2\sigma_{ic}^2}\right), \quad (23)$$

dove  $\mu_{ic}$  e  $\sigma_{ic}^2$  rappresentano rispettivamente media e varianza della feature  $i$  per la classe  $c$ .

A differenza degli altri modelli sottoposti a Grid Search, per Gaussian Naive Bayes non è stata effettuata una ricerca sistematica degli iperparametri. Il classificatore **GaussianNB** è stato valutato mediante 5-Fold Stratified Cross-Validation e successivamente addestrato sull'intero Training Set standardizzato.

### 3.5. Protocollo di validazione

La valutazione dei modelli è stata organizzata separando la fase di selezione e ottimizzazione degli algoritmi dalla valutazione finale della loro capacità di generalizzazione.

Come descritto nella Sezione precedente, il dataset è stato suddiviso in un Training Set contenente 7.903 osservazioni e un Test Set composto da 1.976 osservazioni. Tutte le operazioni di addestramento, selezione degli iperparametri e confronto preliminare tra le configurazioni dei modelli sono state effettuate utilizzando esclusivamente il Training Set. Il Test Set è rimasto invece separato fino alla fase conclusiva dell'esperimento.

Per la selezione dei modelli è stata adottata una 5-Fold Stratified Cross-Validation. Il Training Set viene suddiviso in cinque sottoinsiemi, o *fold*, preservando in ciascuno di essi la distribuzione della variabile target `blueWins`. A ogni iterazione quattro fold vengono utilizzati per l'addestramento e il fold rimanente per la validazione. Il procedimento viene ripetuto cinque volte, in modo che ciascun fold venga utilizzato una volta come insieme di validazione.

La procedura è stata implementata mediante `StratifiedKFold` di `scikit-learn` (?), utilizzando i parametri:

```
n_splits = 5
shuffle = True
random_state = 42
```

La stratificazione è particolarmente utile per garantire che la proporzione tra vittorie del Blue Team e del Red Team rimanga sostanzialmente costante nei diversi fold.

Per Regressione Logistica, Support Vector Machine, Decision Tree, K-Nearest Neighbors e Random Forest, la selezione degli iperparametri è stata effettuata tramite `GridSearchCV`. Per ciascuna combinazione di iperparametri viene calcolata l'Accuracy media ottenuta nei cinque fold di validazione, selezionando quindi la configurazione con il valore medio più elevato.

Gaussian Naive Bayes non è stato invece sottoposto a Grid Search. Il modello è stato valutato direttamente mediante la stessa 5-Fold Stratified Cross-Validation, utilizzando la configurazione standard del classificatore `GaussianNB`.

È importante sottolineare che i risultati ottenuti durante la Cross-Validation non rappresentano la valutazione finale dei modelli. Essi vengono utilizzati esclusivamente per la selezione delle configurazioni da mantenere. In nessuna fase della ricerca degli iperparametri sono state utilizzate osservazioni appartenenti al Test Set.

Una volta completata la selezione dei modelli, i sei classificatori sono stati valutati sulle medesime 1.976 osservazioni del Test Set. Per garantire

coerenza tra addestramento e valutazione, ciascun modello ha ricevuto la stessa rappresentazione dei dati utilizzata durante il training. Regressione Logistica, SVM, KNN, Random Forest e Gaussian Naive Bayes sono stati valutati sul Test Set standardizzato, mentre il Decision Tree è stato applicato alle corrispondenti feature nelle unità di misura originali.

La valutazione finale è stata effettuata utilizzando quattro metriche di classificazione: Accuracy, Precision, Recall e F1-Score. Considerando come classe positiva la vittoria del Blue Team (`blueWins = 1`), l'Accuracy è definita come:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}, \quad (24)$$

dove  $TP$  e  $TN$  rappresentano rispettivamente i veri positivi e i veri negativi, mentre  $FP$  e  $FN$  indicano i falsi positivi e i falsi negativi.

La Precision misura la proporzione di vittorie del Blue Team correttamente identificate tra tutte le osservazioni classificate dal modello come vittorie Blue:

$$Precision = \frac{TP}{TP + FP}. \quad (25)$$

La Recall misura invece la proporzione di vittorie effettive del Blue Team correttamente individuate dal classificatore:

$$Recall = \frac{TP}{TP + FN}. \quad (26)$$

Infine, l'F1-Score rappresenta la media armonica tra Precision e Recall:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}. \quad (27)$$

L'utilizzo congiunto delle quattro metriche consente di confrontare le prestazioni complessive dei modelli e, allo stesso tempo, di evidenziare eventuali differenze nel comportamento rispetto alle due tipologie di errore di classificazione.

A supporto delle metriche aggregate sono state inoltre analizzate le matrici di confusione dei sei classificatori, che permettono di distinguere direttamente il numero di veri positivi, veri negativi, falsi positivi e falsi negativi. Dopo questa valutazione non è stato effettuato alcun ulteriore tuning dei modelli sul Test Set.

## 4. Risultati

La valutazione finale dei sei classificatori è stata effettuata sulle 1.976 osservazioni del Test Set, mantenuto separato durante tutte le precedenti fasi di addestramento e selezione degli iperparametri. I risultati presentati in questa sezione si riferiscono pertanto esclusivamente a dati non utilizzati durante il training dei modelli.

### 4.1. Confronto delle prestazioni predittive

La Tabella 2 riporta Accuracy, Precision, Recall e F1-Score ottenuti dai sei classificatori sul Test Set.

Tabella 2: Prestazioni dei classificatori sul Test Set.

| Modello                | Accuracy      | Precision     | Recall        | F1-Score      |
|------------------------|---------------|---------------|---------------|---------------|
| Random Forest          | <b>0.7247</b> | 0.7251        | 0.7221        | <b>0.7236</b> |
| Regressione Logistica  | 0.7212        | 0.7182        | <b>0.7262</b> | 0.7221        |
| Decision Tree          | 0.7206        | <b>0.7343</b> | 0.6897        | 0.7113        |
| Support Vector Machine | 0.7166        | 0.7156        | 0.7170        | 0.7163        |
| Gaussian Naive Bayes   | 0.7115        | 0.7158        | 0.6998        | 0.7077        |
| K-Nearest Neighbors    | 0.7075        | 0.7056        | 0.7099        | 0.7078        |

La Random Forest presenta il valore più elevato di Accuracy, pari a 0.7247, e il migliore F1-Score, pari a 0.7236. Il modello classifica quindi correttamente circa il 72,5% delle partite del Test Set e mostra il miglior risultato complessivo tra i classificatori considerati.

La Regressione Logistica raggiunge risultati molto vicini, con un'Accuracy pari a 0.7212 e un F1-Score di 0.7221. Essa presenta inoltre il Recall più elevato, pari a 0.7262, indicando la maggiore capacità, tra i modelli analizzati, di identificare le partite effettivamente vinte dal Blue Team.

Il Decision Tree registra invece la Precision più elevata, pari a 0.7343, ma contemporaneamente il Recall più basso tra i modelli principali, pari a 0.6897. Il classificatore risulta quindi più selettivo nel prevedere una vittoria del Blue Team, a costo di non identificare una quota maggiore delle vittorie Blue effettivamente presenti nel Test Set.

SVM, Gaussian Naive Bayes e KNN ottengono prestazioni leggermente inferiori. In particolare, KNN presenta l'Accuracy più bassa, pari a 0.7075.



Nel complesso, le differenze tra i modelli risultano relativamente contenute. La differenza di Accuracy tra Random Forest e Regressione Logistica, i due modelli con il valore più elevato, è di circa 0.35 punti percentuali, mentre la differenza in termini di F1-Score è di circa 0.14 punti percentuali. Questi risultati mostrano quindi che l'aumento della complessità del modello non produce, nel presente esperimento, un incremento molto ampio delle prestazioni predittive.

#### 4.2. Importanza delle caratteristiche

Oltre alle prestazioni predittive, sono state analizzate le informazioni fornite dalla Regressione Logistica e dalla Random Forest circa il peso attribuito alle diverse caratteristiche del gioco.

Per la Regressione Logistica sono stati esaminati i coefficienti del modello addestrato sulle feature standardizzate. A causa della regolarizzazione L1 selezionata durante la fase di ottimizzazione, soltanto quattro feature presentano un coefficiente diverso da zero.

Tabella 3: Feature con coefficiente diverso da zero nella Regressione Logistica.

| Feature            | Coefficiente | Importanza relativa |
|--------------------|--------------|---------------------|
| blueGoldDiff       | 0.9315       | 59.73%              |
| blueExperienceDiff | 0.3916       | 25.11%              |
| blueDragons        | 0.1285       | 8.24%               |
| redDragons         | -0.1080      | 6.93%               |

La variabile nettamente predominante è **blueGoldDiff**, il cui valore assoluto rappresenta circa il 59,7% dell'importanza relativa complessiva dei coefficienti non nulli. Segue **blueExperienceDiff**, con circa il 25,1%.

Il segno positivo dei coefficienti di **blueGoldDiff**, **blueExperienceDiff** e **blueDragons** indica che valori più elevati di queste feature spostano la previsione del modello verso la classe **blueWins** = 1. Il coefficiente negativo associato a **redDragons** opera invece nella direzione opposta.

La Random Forest restituisce una distribuzione meno concentrata dell'importanza tra le feature. Le caratteristiche con il peso maggiore sono riportate nella Tabella 4.

Tabella 4: Principali feature importance della Random Forest.

| Feature            | Feature Importance |
|--------------------|--------------------|
| blueGoldDiff       | 22.24%             |
| blueExperienceDiff | 16.93%             |
| blueTotalGold      | 9.27%              |
| redTotalGold       | 8.77%              |
| blueGoldPerMin     | 7.72%              |
| redTotalExperience | 7.43%              |
| redGoldPerMin      | 6.31%              |

Anche nella Random Forest le due caratteristiche principali sono `blueGoldDiff` e `blueExperienceDiff`, con importanze pari rispettivamente a circa il 22,2% e al 16,9%. Il modello ensemble distribuisce tuttavia una quota rilevante dell'importanza anche su altre variabili economiche e di esperienza, come l'oro totale delle due squadre, l'oro ottenuto al minuto e l'esperienza totale del Red Team.

I due modelli concordano quindi nell'attribuire un ruolo centrale ai differenziali di oro e di esperienza al decimo minuto, pur rappresentando l'informazione disponibile in maniera differente. La Regressione Logistica regolarizzata produce una rappresentazione fortemente selettiva, mentre la Random Forest distribuisce l'importanza tra un insieme più ampio di caratteristiche.

#### 4.3. Matrici di confusione

Per approfondire la natura degli errori di classificazione, sono state analizzate le matrici di confusione dei sei modelli. Considerando come classe positiva `blueWins = 1`, la Tabella 5 riporta il numero di veri negativi (TN), falsi positivi (FP), falsi negativi (FN) e veri positivi (TP).

Tabella 5: Valori delle matrici di confusione sul Test Set.

| Modello                | TN  | FP  | FN  | TP  |
|------------------------|-----|-----|-----|-----|
| Regressione Logistica  | 709 | 281 | 270 | 716 |
| Support Vector Machine | 709 | 281 | 279 | 707 |
| Decision Tree          | 744 | 246 | 306 | 680 |
| K-Nearest Neighbors    | 698 | 292 | 286 | 700 |
| Random Forest          | 720 | 270 | 274 | 712 |
| Gaussian Naive Bayes   | 716 | 274 | 296 | 690 |

La Regressione Logistica produce il maggior numero di veri positivi ( $TP = 716$ ) e il minor numero di falsi negativi ( $FN = 270$ ), risultato coerente con il Recall più elevato osservato nella Tabella 2.

Il Decision Tree presenta invece il maggior numero di veri negativi ( $TN = 744$ ) e il minor numero di falsi positivi ( $FP = 246$ ), ma registra anche il maggior numero di falsi negativi ( $FN = 306$ ). Questo comportamento spiega la combinazione tra Precision elevata e Recall relativamente più basso osservata per questo classificatore.

La Random Forest mostra valori relativamente equilibrati tra le due tipologie di errore, con 270 falsi positivi e 274 falsi negativi, oltre a 720 veri negativi e 712 veri positivi. Tale distribuzione è coerente con il miglior equilibrio complessivo tra Accuracy, Precision, Recall e F1-Score evidenziato nella valutazione finale.

#### 4.4. Analisi critica degli errori e partite borderline

Per verificare se differenti famiglie di classificatori incontrassero difficoltà sulle medesime partite, sono state individuate le osservazioni del Test Set classificate erroneamente da tutti e sei i modelli.

Su 1.976 partite, 391 risultano classificate in modo errato da tutti i classificatori, corrispondenti al 19,79% del Test Set. Gli errori comuni risultano inoltre sostanzialmente bilanciati rispetto alla classe reale: 200 delle 391 partite sono state vinte dal Blue Team e 191 dal Red Team. Non emerge quindi una marcata concentrazione degli errori condivisi su una sola delle due classi.

Per analizzare se tali osservazioni fossero associate a partite economicamente più equilibrate al decimo minuto, è stato confrontato il valore assoluto del differenziale di oro,  $|\text{blueGoldDiff}|$ , tra l'intero Test Set e il sottoinsieme degli errori comuni.

Tabella 6: Confronto del differenziale di oro tra Test Set ed errori comuni.

| Gruppo          | N    | Media $ \text{GoldDiff} $ | Mediana $ \text{GoldDiff} $ | $\leq 1000$ gold |
|-----------------|------|---------------------------|-----------------------------|------------------|
| Intero Test Set | 1976 | 2005.26                   | 1677                        | 31.88%           |
| Errori comuni   | 391  | 1577.96                   | 1367                        | 36.32%           |

Le partite sbagliate contemporaneamente dai sei classificatori presentano un differenziale economico mediamente più contenuto. La media di  $|\text{blueGoldDiff}|$  diminuisce infatti da circa 2.005 gold nell'intero Test Set a

circa 1.578 gold negli errori comuni; anche la mediana diminuisce da 1.677 a 1.367 gold.

Considerando una fascia di 1.000 gold attorno allo zero, il 36,32% degli errori comuni rientra in tale intervallo, rispetto al 31,88% dell'intero Test Set. Ciò indica una maggiore presenza relativa di partite con differenziali economici contenuti tra le osservazioni sbagliate da tutti i modelli.

L'evidenza non consente tuttavia di concludere che gli errori siano concentrati esclusivamente nelle partite caratterizzate da un `blueGoldDiff` prossimo allo zero. Considerando una soglia più restrittiva di 500 gold, infatti, la quota è pari al 14,07% negli errori comuni, leggermente inferiore al 16,04% osservato nell'intero Test Set.

I risultati forniscono quindi un'evidenza parziale a favore dell'ipotesi secondo cui le partite economicamente più equilibrate al minuto 10 risultino più difficili da classificare. Gli errori condivisi presentano nel complesso differenziali di oro più contenuti, ma il solo `blueGoldDiff` non è sufficiente a spiegare tutte le classificazioni errate comuni.

## 5. Discussione

I risultati mostrano che le informazioni disponibili al decimo minuto contengono un segnale predittivo rilevante rispetto all'esito finale della partita. Allo stesso tempo, nessuno dei modelli considerati supera un'Accuracy del 73%, evidenziando come lo stato dell'early game non determini completamente il risultato conclusivo del match.

La presente sezione interpreta tali evidenze dal punto di vista delle dinamiche di gioco, analizza il compromesso tra prestazioni e interpretabilità dei classificatori e discute i principali limiti dell'approccio adottato.

### 5.1. Interpretazione delle evidenze dal punto di vista del gioco

L'analisi dell'importanza delle caratteristiche evidenzia innanzitutto il ruolo centrale delle variabili relative al vantaggio economico e all'esperienza accumulata nei primi dieci minuti. Sia la Regressione Logistica sia la Random Forest identificano infatti `blueGoldDiff` e `blueExperienceDiff` tra le caratteristiche maggiormente associate all'esito finale.

Dal punto di vista del gioco, il differenziale di oro sintetizza la distanza economica esistente tra le due squadre al momento dell'osservazione. Un vantaggio di oro può derivare da diverse dinamiche dell'early game, tra cui

eliminazioni, farming, distruzione di strutture e conquista di obiettivi. Tale risorsa consente inoltre ai giocatori di acquistare oggetti e incrementare le statistiche dei propri campioni. È quindi plausibile che una differenza economica già presente al decimo minuto contenga informazioni utili per distinguere squadre che si trovano in stati di gioco differenti.

Un'interpretazione analoga può essere formulata per **blueExperienceDiff**. Un vantaggio di esperienza rappresenta una differenza nella progressione dei campioni e può riflettere una maggiore acquisizione delle risorse disponibili sulla mappa. Il fatto che questa variabile emerga come seconda caratteristica più rilevante in entrambi i modelli suggerisce che la previsione non dipenda soltanto dall'oro accumulato, ma anche dallo stato di progressione delle due squadre.

Nella Regressione Logistica assumono inoltre un peso diverso da zero **blueDragons** e **redDragons**. La conquista dei draghi rappresenta una forma di controllo degli obiettivi neutrali e può costituire un ulteriore indicatore dello stato della partita. Coerentemente con la codifica del target, il coefficiente positivo associato a **blueDragons** orienta il modello verso la previsione di una vittoria del Blue Team, mentre quello negativo associato a **redDragons** opera nella direzione opposta.

La Random Forest fornisce una rappresentazione più distribuita del segnale predittivo. Accanto ai differenziali di oro ed esperienza, il modello attribuisce infatti importanza anche a variabili quali l'oro totale delle squadre, l'oro ottenuto al minuto e l'esperienza totale. Il risultato suggerisce che il classificatore ensemble utilizzi contemporaneamente diverse descrizioni dello stato economico e della progressione della partita, invece di concentrarsi su un numero ristretto di caratteristiche.

Questi risultati devono tuttavia essere interpretati come *associazioni predittive* e non come relazioni causali. Il fatto che **blueGoldDiff** risulti la caratteristica più importante non consente, ad esempio, di concludere che il solo aumento del differenziale di oro sia la causa della successiva vittoria. Le variabili osservate al minuto 10 sono il risultato di molteplici eventi di gioco tra loro connessi e possono rappresentare indirettamente situazioni più ampie di vantaggio o svantaggio.

Anche l'analisi degli errori comuni fornisce indicazioni in questa direzione. Le 391 partite classificate erroneamente da tutti i modelli presentano un valore assoluto di **blueGoldDiff** mediamente inferiore rispetto all'intero Test Set. Ciò è compatibile con l'idea che situazioni economicamente meno sbilanciate possano offrire ai classificatori un segnale meno netto.

L'evidenza è tuttavia soltanto parziale. Gli errori comuni non sono concentrati esclusivamente nelle partite con differenziale di oro prossimo allo zero e il solo `blueGoldDiff` non permette quindi di spiegare tutte le classificazioni errate. Questo risultato è coerente con la natura dinamica del problema: due partite che al decimo minuto presentano uno stato economico simile possono evolvere successivamente in modi differenti.

### 5.2. *Compromesso tra complessità, interpretabilità e prestazioni*

Il confronto tra i classificatori permette di analizzare uno degli aspetti centrali del presente lavoro: il compromesso tra capacità predittiva e interpretabilità.

La Random Forest ottiene il miglior risultato complessivo, con un'Accuracy pari a 0.7247 e un F1-Score pari a 0.7236. La Regressione Logistica raggiunge tuttavia valori molto vicini, rispettivamente 0.7212 e 0.7221. In termini di Accuracy, il vantaggio assoluto della Random Forest è quindi di circa 0.35 punti percentuali.

L'aumento della complessità del modello produce pertanto, nel presente esperimento, un miglioramento predittivo limitato. La Random Forest può rappresentare relazioni non lineari e combinazioni tra più caratteristiche, mentre la Regressione Logistica assume una struttura decisionale molto più semplice. Nonostante tale differenza, le prestazioni ottenute dai due modelli sul Test Set risultano molto simili.

La Regressione Logistica presenta inoltre un importante vantaggio dal punto di vista interpretativo. La regolarizzazione L1 ha prodotto un modello nel quale soltanto quattro caratteristiche possiedono un coefficiente diverso da zero. È quindi possibile individuare direttamente le variabili utilizzate dal classificatore e osservare anche la direzione della loro associazione con la previsione.

La Random Forest distribuisce invece l'importanza tra un insieme più ampio di feature. Le *feature importance* consentono di ottenere una misura globale del contributo delle variabili, ma non forniscono direttamente il segno o la forma della relazione tra ciascuna caratteristica e la classe prevista. L'interpretazione del meccanismo decisionale complessivo risulta pertanto meno immediata rispetto a quella della Regressione Logistica.

Il confronto non conduce quindi all'individuazione di un modello universalmente preferibile. Se l'obiettivo principale consiste nel massimizzare le prestazioni predittive tra i classificatori analizzati, la Random Forest rappresenta la scelta migliore sulla base del Test Set. Se invece si attribuisce

maggior importanza alla trasparenza della relazione tra caratteristiche e previsione, la Regressione Logistica offre prestazioni quasi equivalenti attraverso una rappresentazione considerevolmente più semplice.

Il risultato costituisce quindi un esempio concreto del compromesso tra complessità e interpretabilità discusso nella letteratura sul Machine Learning interpretabile (Rudin, 2019). Nel caso analizzato, l'incremento di complessità fornito dalla Random Forest determina un vantaggio predittivo misurabile ma quantitativamente contenuto.

È inoltre significativo che entrambi i modelli attribuiscono la massima rilevanza a `blueGoldDiff` e `blueExperienceDiff`. La concordanza tra due algoritmi con strutture differenti rafforza l'evidenza empirica secondo cui lo stato economico e la progressione in esperienza costituiscono componenti particolarmente informative dello stato della partita al minuto 10.

### *5.3. Limiti dello studio*

I risultati ottenuti devono essere considerati alla luce di alcuni limiti legati sia alla rappresentazione delle partite sia al disegno sperimentale adottato.

Il principale limite deriva dal momento nel quale viene effettuata l'osservazione. Il dataset descrive lo stato delle partite esclusivamente al decimo minuto e non contiene informazioni relative alla loro evoluzione successiva. Il classificatore deve quindi prevedere un risultato futuro utilizzando una fotografia parziale del match.

Una squadra in vantaggio al minuto 10 può successivamente perdere tale vantaggio, mentre una squadra inizialmente in svantaggio può recuperare nel corso della partita. Team fight, eliminazioni, errori individuali, cambiamenti nel controllo della mappa e conquista di obiettivi successivi possono modificare sostanzialmente lo stato del match. Queste dinamiche non sono osservabili attraverso le feature utilizzate nel presente studio.

Lo stesso problema riguarda gli obiettivi disponibili nelle fasi più avanzate della partita. Eventi come la conquista o la perdita del Baron Nashor possono influenzare significativamente l'evoluzione del match, ma per definizione non possono essere rappresentati in una fotografia limitata ai primi dieci minuti.

I 391 errori condivisi dai sei classificatori non devono pertanto essere interpretati automaticamente come rimonte o come partite intrinsecamente imprevedibili. L'analisi permette soltanto di affermare che, utilizzando la rappresentazione disponibile al minuto 10, tutti i modelli considerati hanno prodotto una previsione errata sulle stesse osservazioni. Stabilire quali eventi

abbiano determinato l'esito successivo richiederebbe informazioni temporali aggiuntive sull'evoluzione di ciascuna partita.

Un ulteriore limite riguarda le informazioni non rappresentate nelle feature utilizzate. Il modello non dispone, ad esempio, di una descrizione completa della composizione dei campioni selezionati dalle due squadre. Le differenti composizioni possono avere caratteristiche strategiche diverse e presentare differenti potenzialità nelle fasi early, mid e late game. Di conseguenza, due squadre caratterizzate da uno stato quantitativo simile al minuto 10 possono trovarsi in condizioni strategiche non equivalenti.

Analogamente, eventuali eventi non riconducibili alle statistiche considerate, come disconnessioni o problemi tecnici durante la partita, non possono essere identificati dal modello e possono contribuire a determinare un risultato non coerente con lo stato osservato nei primi dieci minuti.

Un'altra limitazione è rappresentata dall'assenza di una dimensione temporale continua. Le feature disponibili sintetizzano lo stato raggiunto al minuto 10, ma non descrivono il percorso attraverso cui tale stato è stato raggiunto. Un differenziale di oro identico potrebbe, ad esempio, derivare da dinamiche temporalmente differenti. Modelli basati su sequenze di osservazioni raccolte a intervalli regolari potrebbero permettere di rappresentare l'evoluzione del vantaggio nel corso dell'early game anziché utilizzare un singolo snapshot.

Infine, i risultati sono necessariamente legati alle caratteristiche del dataset analizzato. League of Legends è soggetto a modifiche periodiche delle meccaniche di gioco, dei campioni e del bilanciamento. In assenza di una valutazione esplicita su partite provenienti da versioni differenti del gioco, non è possibile assumere che le prestazioni osservate rimangano invariate tra patch o periodi temporali differenti.

Tali limitazioni non annullano il valore predittivo osservato, ma definiscono l'ambito entro il quale devono essere interpretati i risultati. Il modello descrive la capacità di utilizzare uno snapshot dello stato della partita al minuto 10 per anticiparne l'esito, senza pretendere di rappresentare tutte le dinamiche strategiche e temporali che determinano il risultato finale.

## **6. Conclusioni e sviluppi futuri**

Il presente lavoro ha analizzato la possibilità di prevedere l'esito finale di partite ranked di League of Legends utilizzando esclusivamente le informazioni disponibili al decimo minuto di gioco. A tale scopo sono stati con-



frontati sei algoritmi di Machine Learning appartenenti a famiglie differenti: Regressione Logistica, Support Vector Machine, Decision Tree, K-Nearest Neighbors, Random Forest e Gaussian Naive Bayes.

L'analisi ha mostrato che le informazioni relative all'early game contengono un segnale predittivo rilevante, pur non essendo sufficienti a determinare con certezza l'esito finale della partita. I risultati permettono quindi di rispondere alle tre domande di ricerca formulate all'inizio dello studio.

**RQ1.** Le informazioni relative allo stato della partita nei primi dieci minuti consentono di anticipare l'esito finale con una capacità predittiva sensibilmente superiore a una classificazione casuale. Il miglior classificatore, la Random Forest, raggiunge infatti un'Accuracy pari a 0.7247 e un F1-Score pari a 0.7236 sul Test Set.

Tali risultati indicano che lo stato dell'early game contiene informazioni utili rispetto all'esito futuro del match. Allo stesso tempo, un'Accuracy inferiore al 73% evidenzia che una parte consistente dell'esito finale dipende da informazioni e dinamiche che non possono essere osservate attraverso il solo snapshot disponibile al minuto 10.

**RQ2.** La Random Forest presenta le migliori prestazioni complessive, ma il vantaggio rispetto alla Regressione Logistica risulta quantitativamente contenuto. La Regressione Logistica raggiunge infatti un'Accuracy pari a 0.7212 e un F1-Score pari a 0.7221, rispetto a 0.7247 e 0.7236 della Random Forest.

L'incremento di Accuracy ottenuto attraverso il modello ensemble è quindi pari a circa 0.35 punti percentuali. Nel contesto analizzato, l'aumento della complessità permette di ottenere un miglioramento predittivo, ma tale miglioramento è limitato rispetto alla maggiore semplicità e interpretabilità offerta dalla Regressione Logistica.

Il Decision Tree rappresenta un ulteriore modello facilmente interpretabile e raggiunge un'Accuracy pari a 0.7206, mostrando anch'esso prestazioni relativamente vicine a quelle della Random Forest. I risultati evidenziano pertanto un compromesso concreto tra massimizzazione delle prestazioni e trasparenza del processo decisionale.

**RQ3.** L'analisi delle osservazioni classificate erroneamente da tutti i modelli fornisce un'evidenza parziale dell'ipotesi secondo cui le partite

economicamente più equilibrate al decimo minuto siano più difficili da classificare.

I sei classificatori commettono simultaneamente un errore su 391 delle 1.976 partite del Test Set, corrispondenti al 19,79%. In questo sottoinsieme, il valore medio di `blueGoldDiff` è pari a circa 1.578 gold, rispetto ai circa 2.005 gold dell'intero Test Set; anche la mediana diminuisce da 1.677 a 1.367 gold.

Gli errori condivisi tendono quindi a presentare differenziali economici complessivamente più contenuti. Tuttavia, essi non sono concentrati esclusivamente nelle partite caratterizzate da un differenziale prossimo allo zero. Il vantaggio economico rappresenta pertanto una componente importante della difficoltà predittiva, ma non è sufficiente da solo a spiegare gli errori dei modelli.

L'analisi dell'importanza delle caratteristiche completa tali risultati. Sia la Regressione Logistica sia la Random Forest attribuiscono un ruolo centrale a `blueGoldDiff` e `blueExperienceDiff`, evidenziando come il vantaggio economico e la progressione in esperienza costituiscano le informazioni maggiormente associate alla previsione dell'esito tra quelle considerate nello studio.

Nel complesso, il lavoro mostra quindi che una fotografia quantitativa dell'early game permette di ottenere previsioni informative sull'esito finale di una partita di League of Legends. Al tempo stesso, la presenza di un numero rilevante di errori condivisi da algoritmi strutturalmente differenti evidenzia i limiti di una rappresentazione basata su un singolo istante temporale.

### *Sviluppi futuri*

Un primo possibile sviluppo consiste nell'estendere la rappresentazione dei dati dalla singola osservazione al minuto 10 a una sequenza di osservazioni raccolte durante lo svolgimento della partita. La disponibilità di informazioni a intervalli regolari permetterebbe di studiare l'evoluzione temporale di variabili quali differenziale di oro, esperienza, eliminazioni e controllo degli obiettivi.

In questo modo sarebbe possibile sviluppare un sistema di previsione *minute-by-minute*, capace di aggiornare progressivamente la probabilità di vittoria al variare dello stato della partita. Un simile approccio permetterebbe inoltre di analizzare in quali fasi del match l'esito diventi progressivamente più prevedibile.

Un secondo sviluppo riguarda l'integrazione di informazioni non presenti nel dataset utilizzato. In particolare, potrebbero essere considerate la composizione dei campioni delle due squadre, le statistiche storiche dei singoli giocatori, i rispettivi ruoli, il livello di esperienza con il campione selezionato e ulteriori informazioni relative alla patch nella quale viene disputata la partita.

La disponibilità di dati temporali più ricchi consentirebbe inoltre di sperimentare modelli progettati per elaborare sequenze. Oltre ai classificatori tradizionali analizzati nel presente lavoro, studi futuri potrebbero confrontare architetture di Deep Learning e modelli sequenziali in grado di apprendere direttamente l'evoluzione dello stato del match.

Tali sviluppi dovrebbero essere accompagnati da un confronto sistematico con modelli più semplici, in modo da verificare se l'eventuale incremento della complessità produca un miglioramento predittivo sufficientemente ampio da giustificare la conseguente riduzione di interpretabilità.

Infine, una validazione su dataset provenienti da periodi temporali, patch e livelli competitivi differenti permetterebbe di valutare in maniera più approfondita la capacità di generalizzazione dei risultati ottenuti e la loro stabilità rispetto all'evoluzione del gioco.

## Riferimenti bibliografici

- Bahrololloomi, F., Klonowski, F., Sauer, S., Horst, R., Dörner, R., 2023. E-sports player performance metrics for predicting the outcome of league of legends matches considering player roles. *SN Computer Science* 4, 238. doi:10.1007/s42979-022-01660-6.
- Breiman, L., 2001. Random forests. *Machine Learning* 45, 5–32. doi:10.1023/A:1010933404324.
- Do, T.D., Wang, S.I., Yu, D.S., McMillian, M.G., McMahan, R.P., 2021. Using machine learning to predict game outcomes based on player-champion experience in league of legends. *arXiv preprint arXiv:2108.02799* doi:10.48550/arXiv.2108.02799.
- Fayyad, U., Piatetsky-Shapiro, G., Smyth, P., 1996. From data mining to knowledge discovery in databases. *AI Magazine* 17, 37–54. doi:10.1609/aimag.v17i3.1230.

- Junior, J.B.S., Campelo, C.E.C., 2023. League of legends: Real-time result prediction. arXiv preprint arXiv:2309.02449 doi:10.48550/arXiv.2309.02449.
- Mitchell, T.M., 1997. Machine Learning. McGraw-Hill, New York.
- Rudin, C., 2019. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nature Machine Intelligence 1, 206–215. doi:10.1038/s42256-019-0048-x.